Reg. No: 2 1 1 6 2 4 1 0 0 1 1 7 4

Question Paper Code: 051071

B.E./B.Tech DEGREE EXAMINATION NOV/DEC - 2025

Third Semester

Computer Science and Engineering

CS23331 DESIGN AND ANALYSIS OF ALGORITHMS

[Common to: AIML, AIDS, CSE-CS, CSD and IT]

(Regulations 2023)

Time: Three Hours Maximum: 100 Marks

Answer ALL Questions

PART A (10x2=20 Marks)

1	What is meant by the Order of Growth? Classify algorithms based on efficiency classes.	[CO1]	[K2]
2	Define Knapsack Problem and specify its objective function.	[CO1]	[K1]
3	State the Substitution Method used for solving recurrence relations.	[CO2]	[K1]
4	What is the main idea behind Greedy Algorithms? Give one real-life application.	[CO2]	[K1]
5	Compare Divide and Conquer with Dynamic Programming techniques.	[CO3]	[K3]
6	Find the recurrence relation for Binary Search and state its order of growth.	[CO3]	[K2]
7	Recall the recursive formula for the Longest Common Substring problem.	[CO4]	[K1]
8	Compare Memoization and Tabulation in Dynamic Programming.	[CO4]	[K2]
9	Name two problems solved using Branch and Bound.	[CO5]	[K1]
10	What is meant by NP-Complete problems? Give one example.	[CO5]	[K2]

PART B (5x13= 65 Marks)

11. a. Illustrate the step-by-step computation of Time Complexity for the following pseudo-code using the counter method and asymptotic notation: [CO1] [K3]

```
Algorithm Matrix Multiply (A, B, n)
Input: Two n × n matrices A and B
Output: Product matrix C = A × B
for i = 1 to n do
  for j = 1 to n do
    C[i][j] = 0
    for k = 1 to n do
      C[i][j] = C[i][j] + A[i][k] * B[k][j]
return C
```

Draw the growth-rate curve and derive its order.

[OR]

b. Using Brute Force, solve the 0/1 Knapsack Problem for items with: Weights = [2, 3, 4, 5], Values = [3, 4, 5, 6], Capacity = 5. Show the decision tree and compute the optimal solution. Discuss the Time Complexity. [CO1] [K3]

12. a. Solve the following recurrence relation using the backward Substitution method and the Master theorem method: $T(n) = 3T(n/2) + n$, $T(1) = 1$. [CO2] [K3]

[OR]

b. Apply the Greedy Algorithm to the Fractional Knapsack Problem with: Weights = [10, 20, 30], Values = [600, 1000, 1200], Capacity = 50. Show all steps with value/weight ratios and final selection and also discuss the other selection procedure that can be applied to solve the above problem. [CO2] [K3]

13. a. Using Divide and Conquer, find the maximum subarray sum for the array [−12, 11, −13, 14, −11, 12, 11, −15, 14]. Show the subarray divisions and merging step values and write the Divide and Conquer Algorithm for the maximum subarray sum problem. [CO3] [K3]

[OR]

b. Discuss in detail about the Merge sort Algorithm and perform Merge Sort on the given array [38, 27, 43, 3, 9, 82, 10]. Illustrate each pass and show the merging process graphically. [CO3] [K3]

14. a. Solve the String Editing (Edit Distance) problem using Dynamic Programming for the two strings:
X = "INTENTION" and Y = "EXECUTION".
Compute the minimum number of insertions, deletions, and substitutions required. Show the DP matrix and trace the optimal path. [CO4] [K3]

[OR]

b. Solve the Staircase Problem using Dynamic Programming, where a person can climb either 1, 2, or 3 steps at a time. Find the total ways to climb 5 stairs and show the DP relation and table. [CO4] [K3]

15. a. Four Wi-Fi access points (APs) are placed at the corners of a square campus block. Adjacent APs interfere if they use the same channel. [CO5] [K3]

- Vertices V = {1, 2, 3, 4} represent APs.
- Edges E = {(1,2), (2,3), (3,4), (4,1)} represent interference between neighboring APs (a 4-cycle).
- Channels (colours) C = {R, G, B}
(e.g., {Ch1, Ch6, Ch11}).

- Goal: assign a channel to each AP so that no adjacent APs share a channel.

Solve the above problem as a Graph Coloring Problem.

[OR]

b. Solve the N-Queens Problem using the Backtracking approach for N = 4. Show all valid configurations step-by-step with a state-space tree. [CO5] [K3]

PART C (1x15=15 Marks)

16. a. Apply the Dynamic Programming approach to solve the Longest Increasing Subsequence (LIS) problem for the array [10, 22, 9, 33, 21, 50, 41, 60]. Construct the DP table, show intermediate updates, and determine the LIS length. [K3]

[OR]

b. For 3 disks, simulate the Tower of Hanoi problem using recursive logic. Show the recursive tree and calculate the number of moves. Discuss an application of Tower of Hanoi in recursive function debugging or memory stack tracing. [K4]

Reg. No : 2 1 1 6 2 3 1 0 0 1 2 5 5

Question Paper Code :201071

B.E./B.Tech DEGREE EXAMINATION NOV/DEC – 2024

Third Semester

Computer Science and Engineering

CS23331 DESIGN AND ANALYSIS OF ALGORITHMS
(Common to IT, AIML, CSD, AI&DS)

(Regulations 2023)

Time : Three Hours | Maximum : 100 Marks

Answer ALL Questions
PART A (10x2=20 Marks)

1.	Given the algorithm: function findMax(a, b) { if (a > b) return a else return b }, determine its time complexity.	[CO1]	[A2]
2.	Calculate the space complexity for a non-recursive algorithm that uses an integer array of size n.	[CO1]	[B1]
3.	Outline the Master Theorem and its application in solving recurrence relations.	[CO2]	[A2]
4.	Which algorithm is primarily used to solve the Minimum Spanning Tree problem and what initial step does it involve?	[CO2]	[A2]
5.	State the complexity of the binary search algorithm.	[CO3]	[A1]
6.	In the context of the Divide and Conquer method, what does the 'conquer step' involve?	[CO3]	[A1]
7.	Outline the recursive formulae for the Rod Cutting problem solved using Dynamic Programming.	[CO4]	[A2]
8.	Define overlapping subproblems.	[CO4]	[A1]
9.	State the primary technique used in the Graph Coloring problem.	[CO5]	[A1]
10.	Name any two typical problems classified as NP-Hard.	[CO5]	[A1]

PART B (5x13= 65 Marks)

11. a. Analyze the provided iterative Fibonacci number algorithm, which is expressed in C-style pseudocode below. Determine both the time and space complexities. Additionally, create a diagram that visually represents the evolution of the fibValues array during each iteration when n = 5. Ensure that each step of the process is clearly depicted. [CO1] [B2]

```
int fibonacci(int n)
{
  if (n <= 1) return n;

  int fibValues[100]; // Array size should be at least 'n+1' in a real
scenario
  fibValues[0] = 0;
  fibValues[1] = 1;

  for (int i = 2; i <= n; i++) {
    fibValues[i] = fibValues[i - 1] + fibValues[i - 2];
  }

  return fibValues[n];
}
```

[OR]

b. Using the Brute Force technique, determine the maximum value obtainable from items that can fit into a knapsack with a capacity of 50 units. Assume item weights are [10, 20, 30] and their corresponding values are [60, 100, 120]. Address the following in your solution: [CO1] [B1]

i. State the Brute force technique applied to solve the Knapsack Problem
ii. List all possible combinations of items that do not exceed the knapsack's capacity and calculate the total value for each combination.
iii. Identify the combination that provides the maximum value without exceeding the capacity limit.
iv. Calculate the space and time complexity of the Brute Force approach for this problem.

12. a. Frame the recurrence relation for a problem where the divide-and-conquer strategy splits the problem into two equal halves, each solved recursively, and combines the results in linear time. Solve this recurrence relation using backward substitution. Provide a detailed step-by-step explanation of the backward substitution process. [CO2] [B1]

[OR]

b. Determine the time complexity of Huffman's algorithm for building optimal prefix codes using a specific example with the set of characters C = {a, b, c, d} and their respective frequencies {5, 9, 12, 13}. Include a diagram that shows the construction of the Huffman tree and explain how it affects the algorithm's time complexity. [CO2] [B1]

13. a. Describe the algorithm and complexity analysis of the Merge Sort algorithm. Explain it in detail with an example. [CO3] [B1]

[OR]

b. Use the Divide and Conquer approach to find the minimum and maximum values in an array. Given the array elements [7, 3, 9, 5, 6, 2, 8], design a Divide and Conquer algorithm to determine the minimum and maximum values. Describe the algorithm steps and draw a diagram illustrating how the array is divided and conquered at each step. [CO3] [B1]

14. a. Using the Dynamic Programming approach, solve the Rod Cutting problem for a rod of 4 meters. Given the prices per meter length as follows: {1: $1, 2: $5, 3: $8, 4: $9}, determine the maximum revenue obtainable. Write the algorithm to determine the maximum revenue, implement it to find the optimal cutting strategy, and analyze the time complexity of your solution. [CO4] [B1]

[OR]

b. Using the Dynamic Programming approach, solve the Longest Common Substring problem for the strings 'fishing' and 'singing'. Write the algorithm to determine the longest common substring, implement it to find the longest substring, and analyze the time complexity of your solution. [CO4] [B1]

15. a. Explain the Graph Coloring problem solved via backtracking with an example showing the color assignment to nodes. [CO5] [B1]

[OR]

b. Discuss the application of approximation algorithms in solving NP-Complete problems using the Traveling Salesman Problem (TSP) as an example. Outline one approximation strategy for TSP, demonstrate how it reduces complexity, and provide an example of how it might be implemented. [CO5] [B2]

PART C (1x15=15 Marks)

16. a.

```
int sumArray(int arr[], int n) {
  if (n <= 0)
    return 0;
  else
    return sumArray(arr, n-1) + arr[n-1];
}
```

[C2]

Given the recursive algorithm above for calculating the sum of an array, perform the following tasks:

a) Frame the recurrence relation for the given recursive function.
b) Solve the recurrence relation using backward substitution to find the time complexity of the algorithm.

[OR]

b. Solve the 0/1 Knapsack problem using the Branch and Bound approach to optimize the educational and recreational resources for a school camping trip. The van can carry up to 100 kilograms. You need to select from the following items, each with a specified weight and 'fun and learning' value: [C2]

- **Tents:** Weight = 20 kg, Value = 40
- **Cooking Equipment:** Weight = 30 kg, Value = 35
- **First Aid Kits:** Weight = 10 kg, Value = 45
- **Educational Kits:** Weight = 25 kg, Value = 60
- **Binoculars:** Weight = 15 kg, Value = 30
- **Water Sports Equipment:** Weight = 40 kg, Value = 50

Your goal is to maximize the total 'fun and learning' value without exceeding the van's weight capacity. Present your solution process, including the use of a decision tree to visualize the branching decisions and the bounds used to prune suboptimal paths.